



RAGs and LangChain Part 2: Post read

Links to Resources Used in the Session

- [Session Resource Document \(Code Snippets & Prompts\)](#)

1.0 Introduction and Conceptual Recap

This session serves as a continuation of Retrieval-Augmented Generation (RAG) fundamentals, moving from basic implementation to advanced data handling, preprocessing, and open-source optimization. The core objective is to understand how to build a RAG pipeline that is cost-effective, accurate, and capable of handling diverse data formats beyond simple text files.

1.1 The Business Case for RAG

In a traditional customer support scenario, human agents must manually search through thousands of policy documents to answer a query. This process is time-consuming (5–20 minutes per query) and prone to error. RAG automates this by allowing an AI to "read" the specific company documentation and retrieve the exact answer in seconds.

1.2 RAG vs. Standard LLMs

Standard Large Language Models (LLMs) like ChatGPT are trained on public web data. They cannot answer questions about private, non-public data (e.g., "What is the leave policy of Company X?").

- **Standard LLM:** Relies on pre-trained "parametric memory" (general world knowledge). It may hallucinate (make up facts) if asked about specific private data.
- **RAG Architecture:** Connects the LLM to a "non-parametric memory" (an external knowledge base). The LLM is instructed to answer *only* using the retrieved context from that knowledge base.

2.0 The RAG Pipeline: End-to-End Architecture

The RAG process involves a specific sequence of seven steps. While the tools may change (e.g., switching from OpenAI to HuggingFace), the architecture remains constant.

Step	Action	Description
1. Loading	Ingestion	Importing data from various sources (PDF, CSV, Docx, Web) into the environment.
2. Preprocessing	Cleaning	Applying NLP techniques (Tokenization, Lemmatization) to clean noise and optimize costs.
3. Chunking	Splitting	Breaking large documents into smaller, manageable segments (chunks) to maintain context windows.
4. Embedding	Vectorization	Converting text chunks into numerical representations (vectors) using embedding models.
5. Storage	Indexing	Storing vectors in a Vector Database (e.g., ChromaDB, FAISS) for efficient searching.
6. Retrieval	Search	Finding the most relevant chunks based on a user's query using semantic similarity.
7. Generation	Synthesis	Passing the retrieved chunks + user query to the LLM to generate a final natural language response.

3.0 Advanced Data Loading

Real-world applications require processing formats beyond simple text files. The session demonstrated handling structured and semi-structured data.

3.1 Handling Word Documents (.docx)

Python libraries such as `python-docx` are used to extract raw text from Word documents. This allows the system to ingest formatted corporate policies, contracts, and internal memos.

3.2 Handling CSV Files (Structured Data)

CSV files (e.g., FAQs, product lists) are structured as tables. The loading process involves:

1. **Reading the File:** Using libraries like pandas to load the dataframe.
 2. **Row Iteration:** Iterating through each row (e.g., Question and Answer columns).
 3. **Serialization:** Converting the row data into a single text string (e.g., "Question: [text] Answer: [text]") so it can be embedded as a single vector.
-

4.0 Natural Language Processing (NLP)

Preprocessing

Preprocessing is critical for optimizing token usage (cost reduction) and improving retrieval accuracy. Raw text often contains noise that does not add semantic value.

4.1 Key NLP Techniques

The session utilized the NLTK (Natural Language Toolkit) library to implement the following:

Technique	Definition	Benefit in RAG
Tokenization	Breaking text into individual units (words or sub-words).	Allows the model to process text mathematically. Essential for counting costs.
Stop Words Removal	Removing common helping verbs (is, the, are, for) that carry little semantic meaning.	Reduces the number of tokens sent to the LLM, directly lowering API costs without losing core meaning.
Lemmatization	Converting words to their base/root form (e.g., "running" → "run", "better" → "good").	Standardizes text so that variations of a word match the same search query, improving retrieval recall.
Lowercasing	Converting all text to lower case.	Ensures "Apple" and "apple" are treated as the same entity during search.

4.2 The Cost-Benefit Analysis

While preprocessing reduces costs (fewer tokens), aggressive cleaning can sometimes strip necessary context. However, modern LLMs are robust enough to reconstruct sentences even if stop words are removed (e.g., "Company policy 12 days leave" is understood as "The company policy provides 12 days of leave").

5.0 Advanced Chunking Strategies

Chunking determines how the AI "reads" the document. If chunks are too small, context is lost. If too large, they confuse the model or exceed token limits.

5.1 Five Major Chunking Strategies

1. **Sentence-Based:** Splits text after a specific number of sentences (e.g., every 5 sentences). Good for FAQs but may break narrative flow.
2. **Paragraph-Based:** Splits text at new lines or paragraph breaks. Preserves structural context.
3. **Token-Based (Industry Standard):** Splits text based on a strict token count (e.g., 500 tokens). This is the most precise method for managing LLM context windows.
4. **Header-Based:** Uses document structure (H1, H2) to create chunks. Excellent for structured documentation where sections are logically distinct.
5. **Recursive Character Splitter (with Overlap):** Splits by characters but respects delimiters (paragraphs, then sentences, then words). Crucially, it includes **overlap**.

5.2 The Importance of Overlap (Sliding Window)

Concept: When chunking, the end of Chunk A is repeated as the beginning of Chunk B.

Why? Without overlap, a sentence might be cut in half ("The policy for sick leave is..." | "...12 days per year"). Overlap ensures the complete thought is preserved in at least one chunk, maintaining semantic continuity.

6.0 Embeddings and Vector Stores

This session introduced open-source alternatives to OpenAI, allowing for free and private implementations.

6.1 Embedding Models

Embeddings convert text into high-dimensional vectors (lists of numbers) representing semantic meaning.

- **OpenAI (text-embedding-3-small/large):** High accuracy, paid service, data leaves local environment.
- **Open Source (HuggingFace):**
 - **MiniLM:** Lightweight, fast, free. Good for general use.
 - **BGE (BAAI General Embedding):** High performance on benchmarks.
 - **E5:** Another strong open-source contender.

Implementation Note: These can be run locally using the SentenceTransformer library, ensuring data privacy and zero API cost.

6.2 Vector Databases: ChromaDB vs. FAISS

Vector databases store the embeddings and perform the "Nearest Neighbor" search to find relevant chunks.

Feature	ChromaDB	FAISS (Facebook AI Similarity Search)
Developer	Chroma (Open Source)	Meta (Facebook) Research
Speed	Moderate to Fast	Extremely Fast (Optimized for density)
Scale	Good for standard applications	Excellent for millions/billions of vectors
Use Case	General RAG applications	High-latency sensitivity, massive datasets

7.0 Retrieval and Generation Optimization

7.1 Retrieval Metrics

- **Cosine Similarity:** Measures the angle between the query vector and document vectors.
 - **High Similarity (near 1):** Vectors point in the same direction (meanings are similar).
 - **Low Similarity (near 0 or negative):** Vectors are unrelated or opposite.

- **Top-K Retrieval:** Determining how many chunks to retrieve (e.g., Top 3 or Top 5). Retrieving too many increases cost and noise; retrieving too few risks missing the answer.

7.2 Re-Ranking

Sometimes the initial vector search retrieves relevant documents but orders them poorly.

Re-ranking involves a second pass using a more precise model (Cross-Encoder) to re-order the top retrieved chunks, ensuring the LLM sees the most relevant information first.

7.3 Prompt Engineering for RAG

The final prompt sent to the LLM must be structured carefully to prevent hallucinations.

Structure of a RAG Prompt:

1. **Role Definition:** "You are a helpful assistant for Company X."
 2. **Constraint:** "Answer ONLY using the provided context. If the answer is not in the context, state that you do not know."
 3. **Context Injection:** [Insert Retrieved Chunks Here]
 4. **User Query:** [Insert User Question Here]
 5. **Citations:** "Cite the source document and page number for your answer."
-

8.0 Summary and Key Takeaways

- **Flexibility:** RAG is not limited to PDFs. It can ingest CSVs, Word docs, and web data.
 - **Cost Optimization:** NLP techniques like stop-word removal and tokenization reduce the payload sent to LLMs, saving money without sacrificing significant accuracy.
 - **Open Source Viability:** You do not need expensive OpenAI subscriptions for the entire pipeline. Libraries like SentenceTransformer (embeddings) and FAISS (storage) allow for free, local, and private RAG implementations.
 - **Context is King:** The "Sliding Window" (overlap) chunking strategy is essential to prevent cutting information in half.
 - **Latency Matters:** For production environments, choosing the right vector store (like FAISS) can significantly reduce the time-to-answer.
-

9.0 Self-Check Questions

1. **Conceptual:** Explain the difference between "Parametric Memory" (Standard LLM) and "Non-Parametric Memory" (RAG). Why is this distinction important for corporate data?
 2. **Process:** Why is "Overlap" or "Sliding Window" recommended during the chunking phase? What problem does it solve?
 3. **Optimization:** How does removing "Stop Words" during preprocessing affect the financial cost of running a RAG pipeline?
 4. **Architecture:** Compare FAISS and ChromaDB. In which scenario would you specifically choose FAISS over ChromaDB?
 5. **Implementation:** If a user asks a question that is *not* covered in the provided documents, how should a properly configured RAG system respond? Why?
 6. **Data Handling:** Describe the process of converting a structured CSV file into a format suitable for vector embedding.
-

10.0 Additional Learning Resources and Practice

- **Python Libraries to Explore:** langchain, tiktoken, nltk, sentence-transformers, faiss-cpu, pandas.
- **No-Code Alternatives:** For those uncomfortable with Python, tools like **n8n** and **Make.com** offer drag-and-drop nodes to build similar RAG pipelines.
- **Practice Project:** Create a simple RAG system that ingests a CSV of "Frequently Asked Questions" and answers user queries using the FAISS vector store and an open-source embedding model.